Person 2 Script — Workflow Automation

Hammad · Local Password Manager launch · Asana workspace “Local Password Manager” · about 5 minutes. **Say** = your words. Orange = what you click. Grey boxes = background so you understand it and can answer follow-up questions; you don't read them out.

Part 1 — Group assignment (about 90 seconds, one block each)

Same script for all three of us. **Your lines are in blue.**

Abbas (scenario)
“Our company is preparing the launch of a new product: the Local Password Manager v1.0. Product Management, Design, Marketing and Sales have to coordinate their work, requirements and priorities change, stakeholders need regular updates, and repetitive work should be automated. We built one connected example in Asana to show how an agile team handles that.”

Hammad (you) (vision and goal)
“Our **Product Vision**: for students and individuals who manage many online accounts but don’t want their passwords in someone else’s cloud, our Local Password Manager is a simple, offline vault that keeps all credentials encrypted on the user’s own device behind one master password. Our **High-Level Product Goal**: launch version 1.0 on 15 January 2027 with zero critical security findings, and reach 500 student users within 8 weeks of launch.”

Adebowale (direction)
“The vision tells us why we build it and for whom, so it helps us say no: for example, no cloud sync in version 1.0. The goal makes the vision measurable: a fixed date, a quality bar and a user target. Every task and request is prioritised by how much it helps that goal. Both are documented in the overview of our Password manager MVP project and in Asana Goals.”

Abbas (value and principle)
“We chose the Agile value **responding to change over following a plan**, because a launch with four departments always meets new requirements. And the principle **business people and developers must work together daily**, because no department can launch alone. Adebowale shows how we coordinate change, Hammad shows how we automate it, and I show how it connects to our strategic goal.”

Who shows what
• Person 1, Adebowale: cross-functional collaboration, in *v1.0 Launch — Cross-functional Plan*
• **Person 2, Hammad: workflow automation, in *Launch Requests***
• Person 3, Abbas: portfolio and strategic alignment, in *Goals* and the portfolio *Password Manager Launch 2027*

Adebowale presents Person 1, then hands over to you.

Part 2 — Your demo

Before you start: have the project *Launch Requests* open on the **Board** tab, and the form open in a second browser tab, so you never search for anything live.

1. Opening

Click: nothing — just talk.

Say:

“Thank you, Adebowale. You just saw a new request handled by hand. Our scenario says the team spends too much time manually assigning tasks, following up on requests and updating workflows. I'll show how we automate exactly that in our project Launch Requests.”

What this means (so you can explain it in your own words): You link your part to Adebowale's. He showed people coordinating by hand (comments, mentions, reassigning). You show the same kind of request handled automatically. Launch Requests is a separate “intake” project: it's where requests from Marketing, Sales and Design arrive before they become real work. Keeping intake separate keeps the main launch plan clean.

2. Workflow stages

Click:

1) Left sidebar → **Launch Requests**.
2) Top tabs → **Board**.
3) Move your mouse slowly across the columns left to right while you name them.
4) Point at the task *Password-strength meter* in **Parked**.

Say:

“Every launch request follows the same workflow: New Requests, Triage, In Progress, Review and Approval, and Done. Parked holds good ideas outside version 1.0 — like this password-strength meter — because our vision says the secure, offline core comes first.”

“This is a **Kanban** flow: requests arrive at any time, so work keeps flowing instead of waiting for the next sprint. The product itself is built in Scrum sprints in our MVP project, and product requests reach its backlog through Triage.”

What this means (so you can explain it in your own words): Each column is a **section** = one stage of the process. **New Requests**: just arrived, nobody has looked yet. **Triage**: someone checks if the request is clear and worth doing now. **In Progress**: being worked on. **Review / Approval**: the Product Owner checks it. **Done**: finished. **Parked**: not now, but not forgotten. This is basically a **Kanban board**: everyone sees the flow, and work is pulled from left to right. Agile link: Parked lets us respond to change without losing ideas.

3. Custom fields

Click:

1) Top tabs → **List**.
2) Point at the column **Request Type**, then at **Priority**.
3) Optional: click the Priority column header → sort, to show High requests jump to the top.

Say:

“Each request has two custom fields. Request Type says what kind of work it is — design asset, marketing content, sales material, product change or security issue. Priority says how urgent it is. These fields matter because the automation reads them.”

What this means (so you can explain it in your own words): A custom field is an extra, structured column on every task, like a dropdown in a spreadsheet. Free text such as “can someone do a poster?” can't be read by a rule, but a dropdown value like “Design asset” can. So custom fields are the **foundation** of the automation. Priority helps the Product Owner order the work, like ordering a product backlog. **If asked “why not just use tags?”** — tags are free and messy; a dropdown field has fixed options, so it's consistent and rules can rely on it.

4. Task template — show it being used, not just stored

Goal of this step: show that one click creates a complete, ready-to-work task. That is what “reusable workflow structure” means.

Main way (use the template live):

1) Stay on the **List** tab of Launch Requests.
2) Top left, next to the blue **+ Add task** button, click the small **down arrow (v)**. A menu opens with the project's task templates.
3) Click **Marketing Asset Request**. A new task opens on the right, already filled in.
4) Type a name: *Demo – Instagram launch post*.
5) Point at the **description**: Goal, Target audience, Definition of Done.
6) Scroll down in the task to **Subtasks** and point at the four subtasks one by one.
7) Close the task with the **X** (top right of the task panel).

Backup way (if the down arrow isn't there): top right **Customize** → **Task types and templates** → **Task** → **Templates** → click **Marketing Asset Request**. This opens the template itself; show the description and subtasks there, then close it.

Say (while doing steps 2–3):

"Marketing requests always need the same steps, so instead of writing them again every time, we made a task template. Watch: one click on Marketing Asset Request…"

Say (while doing steps 5–6):

"…and the task already has the goal, the target audience — students who don't trust the cloud — and a Definition of Done. Below are four subtasks: draft copy, design the visual, a feature accuracy check by the Product Owner, and publish. Nobody has to remember the steps, nothing gets skipped, and every marketing asset has the same quality."

Say (closing the task):

"That's a reusable workflow structure: the process is built into the task itself."

What this means: a template is a "copy-paste master" for a task. It removes repetitive setup (manual work), which is exactly your topic. The **feature accuracy check** subtask matters for our product: it stops Marketing from promising something we don't do, e.g. "cloud sync", which would break our vision. The **Definition of Done** is an agile idea: everyone agrees in advance when something counts as finished. **Heads-up:** creating this task may also trigger rule R1 (task added) and it may jump to Triage on its own. If that happens, smile and say "and that's already our first rule working — I'll show it in a second." Delete the demo task after the presentation. **Practise this step once before the exam** so you know exactly where the arrow is.

5. Form — where it lives, then submit it live

Important: there is **no "Forms" tab** in the top row (Overview, List, Board, …). In our project the form is found in two places. Show place A, mention place B, then submit from the form link.

Before the exam (preparation): open this link in a **second browser tab** and leave it there:
(form link not published here; open it from Customize → Forms in Asana)

Place A — the form icon on the section (show this):

1) Launch Requests → **List** tab.

2) Find the section header **New Requests** (inbox emoji). Right next to the name there is a small **page icon** (it looks like a sheet of paper with lines). That icon means "a form feeds into this section".

3) Click the icon → a small menu opens: **Forms for New Requests** → **Launch Request Form**.

Place B — Customize (just mention it, or use it as backup):

Top right **Customize** → in the panel click **Forms** (it shows "1") → **Launch Request Form**. This opens the form editor, where you could point at the questions and the "Required" switches.

Submit live:

4) Switch to the second browser tab with the form link.

5) Fill in all five required fields: **Name**: *Partner pricing sheet* (this becomes the task name) · **Email address**: your own email · **What type of request is this?**: **Sales material** · **What do you need?**: *A one-page price sheet for partner universities* · **Priority**: **Medium**.

6) Click **Submit**, then switch back to the Asana tab.

Say (while showing the icon, steps 1–3):

"Requests used to arrive by email, chat and in meetings, and some got lost. So we built one form. This little icon next to New Requests shows it: everything submitted through the Launch Request Form lands in this section."

Say (while filling it in, steps 4–6):

"Every department uses this one form, so it's our single front door. Request Type and Priority are required, so the automation always has the information it needs. I'll submit a request as Sales now: a partner pricing sheet, sales material, medium priority."

What this means: the form is the **single front door** (in agile terms: one intake channel for the backlog). Each submission automatically becomes a task in **New Requests**, and the answers fill the custom fields Request Type

and Priority. Making those two questions **required** is the key design choice: rules can only work if the input is complete. Agile link: business people (Sales, Marketing) reach the team daily through the same channel, which is our principle. **If asked "why a form and not just creating tasks?"** — a form forces the same structure every time, people outside the project can use it, and it triggers the automation automatically. **If the form tab got closed:** use place B (Customize → Forms → Launch Request Form) to open it again.

6. Rule R1 in action

Click:

1) Go back to the Launch Requests tab → **Board**.
2) Find *Partner pricing sheet* in the **Triage** column and click it.
3) Point at the **assignee** (you).
4) Scroll to the bottom of the task to the **activity log** and point at the lines from **Asana Rule**.

Say:

"Nobody touched this task. It's already assigned to me and moved to Triage. The activity log shows Asana Rule did it — within seconds."

What this means (so you can explain it in your own words): This is the proof that the automation really works live. The activity log is like a history: it records who did what. Here it says "Asana Rule" instead of a person. If it takes a few seconds, keep talking about the form — rules can take up to a minute. **Backup:** if it hasn't moved yet, open an older test task or go straight to step 7 and explain the rule.

7. Rule R1 explained

Click:

1) Top right **Customize** → under Rules/Automations click **R1 - Auto-assign by Request Type**.
2) Point at the trigger, then the conditions, then the actions.

Say:

"Every rule has a trigger, a condition and an action. The trigger: a task is added. The condition: which Request Type it has. The action: assign the right owner and move it to Triage. Design assets go to Adebowale, product changes and security issues to Abbas as Product Owner, marketing and sales material to me. It goes to Triage, not straight to In Progress, because a person still checks each request — automation removes admin, not judgement."

What this means (so you can explain it in your own words): Think of a rule as **"WHEN … IF … THEN …"**. WHEN a task is added, IF Request Type = Design asset, THEN assign Adebowale. This is **automatic assignment based on category**, exactly the first example in our assignment sheet. Security issues go to Abbas because security is critical for our goal (zero critical security findings). We deliberately stop at **Triage**: a human still decides whether to start it. That shows we thought about the risk of over-automation.

8. Rule R2

Click:

1) Close the rule with **X**.
2) Board → tick the checkmark on *Partner pricing sheet* to mark it complete.
3) Watch it move to **Done**.

Say:

"The second rule: when a task is marked complete, it moves to Done automatically. So the board always shows the truth, and nobody has to remember to move cards. So before, someone had to read, assign and move every request by hand. Now the form collects it, R1 routes it and R2 closes it, and the team spends its time on the actual work."

What this means (so you can explain it in your own words): R2 = **automatically moving a task to another workflow stage**, the second example in the assignment. Without it, people often tick a task complete but forget to move the card, so the board lies. With R2 the board is always correct, which makes monitoring reliable. Together R1 + R2 = the required "at least two working automations". Manual steps that disappeared: collecting requests from email/chat, typing them in, deciding the owner, assigning, moving to the next stage, moving to Done.

9. Monitoring

Click:

1) Top tabs → **Dashboard**.
2) Point at the chart **tasks by section** and at the bar for **Review / Approval**.
3) Optional: go back to Board and point at the three cards waiting in Review / Approval.

Say:

“The dashboard shows where work piles up. Three tasks are waiting in Review and Approval — the blog post, the social media visuals and the app store screenshots. So our bottleneck is approvals, not the work itself. More automation wouldn't fix that; a daily approval slot for the Product Owner would.”

What this means (so you can explain it in your own words): A **bottleneck** is the stage where work queues up, like a traffic jam. You spot it when one column/bar is much bigger than the others, or tasks stay there too long. The smart point: not every problem is solved with more automation. Here the fix is a **people/process** change. Agile link: this is like limiting work in progress in Kanban and inspecting the flow.

10. Limitation

Click: nothing — just talk.

Say:

“One limitation: automation is only as good as its input. If someone picks the wrong Request Type, the rule confidently assigns the task to the wrong person, and nobody notices. Too many rules can also create notification noise.”

What this means (so you can explain it in your own words): Rules don't think; they follow the field value. Wrong input = wrong output. Other risks you can mention if asked: rules are hidden, so new team members don't know why things happen; automating an unstable process just makes the mess faster; and too much automation can reduce flexibility.

11. Improvement

Click: nothing — just talk.

Say:

“To improve the workflow, I'd add a rule that moves reopened tasks back to In Progress, add a daily approval slot to clear the bottleneck, and review all rules in every sprint retrospective.”

What this means (so you can explain it in your own words): This is **continuous improvement**. The retrospective is the Scrum event where the team asks “what should we change?”, so it's the natural place to check whether rules still fit. Another idea if asked: a rule that reminds the Product Owner when a task sits in Review for more than two days.

12. Agile value and principle

Click: nothing — just talk.

Say:

“This is responding to change over following a plan: a new or changed request reaches the right person in seconds instead of days. And it supports business people and developers working together daily: Sales and Marketing use the same form and workflow as the product team.”

What this means (so you can explain it in your own words): Here you connect your demo to the group's shared value and principle. This is required by the assignment, so don't skip it even if time is short.

13. Reflection

Click: nothing — just talk.

Say:

“So which automation features give agile teams the greatest value? **Forms, custom fields and rules working together.** The form is one front door for every request, so nothing gets lost in emails or chats. Custom fields like Request Type and Priority give each request a structure. And rules act on those fields

instantly: in our project, R1 assigns each request to the right owner and moves it to Triage, and R2 moves finished work to Done. Together they remove repetitive admin, keep the board always accurate, and let the team respond to change within seconds. Task templates add value too, because recurring work like a marketing asset always starts with the same steps and Definition of Done."

"Before automating a workflow, a team should consider four things. **One, is the process stable?** Don't automate a process that's still changing, or you just speed up the mess. **Two, is the input reliable?** Rules only follow the fields; if someone picks the wrong Request Type, the task goes to the wrong person — that's why our form questions are required. **Three, do people keep the important decisions?** Approvals and prioritisation stay human — that's why R1 moves tasks to Triage, not straight to In Progress. **Four, will the rules be reviewed regularly?** Too many rules create notification noise and hidden logic, so we review ours in every retrospective."

> **What this means (so you can explain it in your own words):** This answers the official reflection question. **Short version if time is tight:** "Forms, fields and rules together, because they turn messy requests into an instant, consistent flow. Before automating, check that the process is stable, the input is reliable, humans keep key decisions, and the rules are reviewed." **Memory trick: Door → Data → Decision** (form → fields → rules). The four checks: **Stable, Reliable input, Humans decide, Review**.

14. Hand over

Click: nothing — just talk.

Say:

"Work now flows efficiently. Abbas will show whether it moves us toward our launch goal."

> **What this means (so you can explain it in your own words):** Short and clean. You hand over from the daily workflow (your part) to strategy (Abbas's part).

Abbas presents Person 3, then closes for the group:

Group closing — Abbas

"From one request in a form to a company goal: that is how Asana connects daily work to strategy for our launch."

Which agile method we use (for the Q&A)

One-sentence answer:

"A hybrid: Scrum for building the product, Kanban for the continuous stream of launch requests, and a portfolio with Goals to connect everything to the launch goal."

Where it is in Asana (written in each project's **Overview** tab under "Agile method"):

• **Scrum — Password manager MVP** (Abbas, Product Owner). Product Backlog → *Sprint 1 - To Do* → In Progress → Review/Testing → Done. Sprint Goal in Overview point 10, user stories with acceptance criteria, QA-01 as Definition of Done.
• **Kanban — Launch Requests** (you). No sprints; requests flow continuously, work is pulled left to right, the dashboard shows bottlenecks.
• **Cross-functional board — v1.0 Launch Plan** (Adebowale). Four departments, dependencies, approvals, milestones.
• **Portfolio + Goals — Password Manager Launch 2027** (Abbas). Connects all three projects to the launch goal.

Scrum vs Kanban in one line each: Scrum = fixed-length sprints with a sprint goal, a Product Owner and set events. Kanban = no sprints, continuous flow, visualise the stages and limit work piling up.

Why hybrid? Building software needs a planned, time-boxed goal (Scrum). Requests from Marketing, Sales and Design arrive any day and can't wait two weeks for the next sprint (Kanban). Triage protects the sprint: only the Product Owner decides if a request enters the backlog.

Scrum events in Asana: Sprint Planning = move tasks into Sprint 1 - To Do · Daily Scrum = look at the board together · Sprint Review = status update · Retrospective = mostly outside Asana; we use it to review the rules and the approval bottleneck.

Extra Q&A questions

Q: How does automation improve transparency and help in evolving the workflow?

Short answer: Automation makes every step visible and consistent, and that visibility shows us what to improve next.

Transparency: every request goes through the same form and the same stages, so everyone sees the same truth. R1 shows instantly who owns a request, R2 keeps the board up to date, and the activity log records every action, including “Asana Rule”. Nothing is hidden in emails or chats.

Evolving the workflow: because the board and dashboard are always correct, problems become visible. In our project the dashboard showed two tasks stuck in Review / Approval, so the bottleneck is approvals. In the retrospective we change the process (a daily approval slot) or add a rule (move reopened tasks back to In Progress). Rules are easy to change, so the workflow improves step by step — inspect and adapt.

Our example: “Partner pricing sheet” is submitted, R1 assigns it to me and moves it to Triage, and the activity log shows exactly when and how.

Q: Which Scrum events are best on Asana?

Short answer: Sprint Planning and Sprint Review work best in Asana. The Daily Scrum is supported. The Retrospective mostly happens in conversation.

• **Sprint Planning — best fit.** The Product Owner orders the Product Backlog, and the team moves stories into *Sprint 1 - To Do* with owners, due dates and priority. The Sprint Goal is written in the MVP Overview.
• **Sprint Review — best fit.** Status updates, the dashboard and the portfolio show stakeholders what is done and what is at risk (e.g. the MVP status “at risk: QA checklist not started yet”).
• **Daily Scrum — supported.** The team looks at the board together: what's in progress, what's blocked (hourglass = dependency). The conversation itself still happens face to face.
• **Retrospective — weakest fit.** It's about people talking openly; Asana has no built-in retro board. We only record the agreed actions as tasks, e.g. “review the automation rules”.

Why: Asana is strongest at **planning and visibility** (backlog, boards, status, dashboards). It is weaker at the **human conversation** parts of Scrum, and it has no native sprint features like burndown charts or story points, which Jira has.

Q: What are the best uses for Asana?

Short answer: Asana is best for **cross-functional work that needs to be organised, visible and partly automated** — especially when several departments work towards one goal.

• **Product launches and marketing campaigns:** many departments, many handovers, one deadline — exactly our v1.0 launch.
• **Request intake and repetitive processes:** forms + rules + templates for requests, approvals and onboarding (our Launch Requests).
• **Cross-team coordination:** shared projects, dependencies, comments and approvals instead of email chains.
• **Lightweight agile work:** Kanban boards, backlogs and simple sprints for teams that are not purely software developers.
• **Strategic alignment and reporting:** Goals, Portfolios, Timeline and dashboards to show management progress across projects.
• **Mixed teams with non-technical members:** it's easy to learn, so Marketing and Sales actually use it.

One-line rule: use Asana to **coordinate** work across people and teams; use specialised tools (e.g. Jira, MS Project) for deep software development or heavy budget and resource planning.

Q: Where is Asana not helpful? In which areas of project management is it weak?

Short answer: Asana is great for organising, automating and showing work, but weak for deep software development, detailed budgets and resources, strict traditional planning, and the human side of a team.

• **Software development / Scrum detail:** no native story points, burndown charts, bug tracking or code links. *Better:* Jira (with GitHub/GitLab).
• **Cost and budget management:** no real budgeting, invoicing or earned-value tracking. *Better:* Excel, MS Project, ERP tools.
• **Resource and capacity planning:** Workload view exists, but only on paid plans and it's basic. *Better:* MS Project, Float.
• **Traditional / waterfall projects:** no critical path, baselines or Gantt detail for large regulated projects (e.g. construction). *Better:* MS Project, Primavera.
• **Risk management:** no built-in risk register or risk scoring; we had to model risks as tasks (SEC-01).
• **Communication and team culture:** it can't replace real conversations, retrospectives or conflict resolution. *Better:* meetings, Slack/Teams, Miro for retros.
• **Very small or one-off tasks:** the setup is overhead; a simple to-do list or Trello is enough.

For our project specifically: the actual coding of the password manager would live in Jira + GitHub, not Asana. Many key features (Goals, Portfolios, Timeline, Rules, Forms) need a paid plan or trial. And a security-focused product should note that Asana itself is a cloud tool, so no real passwords or secrets should ever be stored in tasks.